Por qué migrar de PHP a Ruby on Rails API

Fecha: 2026-04-10
Audiencia: Equipo técnico, stakeholders

Resumen ejecutivo

PHP nunca fue el backend real de TTPN. Fue un intermediario temporal que conectaba la app móvil a la base de datos que Ruby construyó. Mantenerlo activo tiene un costo creciente en seguridad, mantenimiento y velocidad de desarrollo. Este documento argumenta por qué eliminarlo es urgente, no opcional.

---

1. PHP nunca fue el backend — Ruby sí

Ruby on Rails creó y controla toda la infraestructura de datos:

Componente	Creado por
Todas las tablas de la BD	Ruby (migrations)
Triggers PostgreSQL (`sp_tctb_insert`)	Ruby
Funciones PG (`buscar_booking`, `buscar_booking_id`, `buscar_nomina`)	Ruby
Índices y constraints	Ruby
Lógica de negocio (nómina, cuadre, discrepancias)	Ruby

PHP solo ejecuta SQL directamente contra esa base de datos. Es un proxy delgado sin lógica propia. Si PHP desaparece mañana, la base de datos no pierde nada — solo se queda sin un canal de acceso que debe ser reemplazado por Rails.

---

2. Vulnerabilidades de seguridad activas en PHP

2.1 Autenticación falsa

PHP no verifica contraseñas en el servidor. La comparación de credenciales ocurre **en el cliente Android**:

```
// La app descarga el hash de la BD y lo compara localmente con BCrypt
// Cualquiera con acceso a la red puede interceptar el hash y compararlo
offline
```

Impacto real: Un atacante que intercepte tráfico HTTP obtiene el hash BCrypt y puede intentar un ataque de diccionario offline sin límite de intentos.

Rails tiene: JWT firmado, autenticación server-side, rate limiting, tokens de sesión con expiración.

2.2 SQL sin parametrizar (riesgo de inyección)

El PHP construye queries concatenando strings directamente desde los parámetros de la app:

```
// Patrón típico en ttpn_php/:
$query = "INSERT INTO travel_counts VALUES(" . $_POST['id'] . ", ...)";
```

Impacto real: Cualquier campo que la app envíe puede contener SQL malicioso que se ejecute directamente en Supabase/PostgreSQL.

Rails tiene: ActiveRecord con parámetros siempre escapados automáticamente.

2.3 IDs generados en el cliente

La app Android hace `GET max_id + 1` antes de cada INSERT. Esto significa:

- Dos dispositivos simultáneos pueden calcular el mismo ID → colisión silenciosa
- Un atacante puede manipular el ID enviado para sobrescribir registros existentes

Rails tiene: Autoincrement de PostgreSQL — el servidor asigna el ID, el cliente lo recibe en la respuesta.

2.4 Credenciales de BD expuestas en el servidor PHP

El archivo de conexión PHP tiene usuario y contraseña de Supabase en texto plano en el servidor Heroku. Si el servidor PHP es comprometido, la BD de producción queda expuesta directamente.

Rails tiene: Variables de entorno, conexión pooling, nunca expone credenciales al cliente.

---

3. Funcionalidades que PHP no puede dar y Rails sí

Funcionalidad	PHP actual	Rails API
Autenticación JWT con expiración	❌ No existe	✅ Implementado
Control de roles y privilegios	❌ No existe	✅ Tabla `privileges` + `role_privileges`
Auditoría de quién hizo qué	❌ No existe	✅ `created_by_id`, `updated_by_id` en todas las tablas
Validaciones de negocio	❌ No existe	✅ Modelos ActiveRecord con validaciones
Callbacks automáticos (discrepancias, nómina)	❌ Parcial, inconsistente	✅ Callbacks y triggers coordinados
Versionado de API (`/api/v1/`)	❌ No existe	✅ Namespacing por versión
Rate limiting	❌ No existe	✅ Configurable por endpoint
Logs estructurados	❌ No existe	✅ Rails logger + Sidekiq

Funcionalidad	PHP actual	Rails API
Workers en background (Sidekiq)	❌ No existe	✅ Para notificaciones, cuadres diferidos
Tests automatizados	❌ No existe	✅ RSpec + CI posible

---

4. El costo de mantener PHP activo

4.1 Doble mantenimiento

Cada vez que se agrega una regla de negocio al backend Rails (ejemplo: `buscar_nomina` en el trigger), hay que verificar que PHP también la implemente — o quedan dos caminos de datos con comportamiento diferente.

Ejemplo real identificado:

- Ruby trigger calcula `viaje_encontrado` y `ttpn_booking_id` ✅
- PHP calcula lo mismo de forma independiente ✅
- Ruby trigger **no** asignaba `payroll_id` ❌
- PHP sí lo asignaba ✅
- Resultado: los travel_counts creados desde Rails API quedaban sin `payroll_id` — bug silencioso en producción que tomó análisis profundo identificar.

4.2 Lógica de negocio dispersa en 3 lugares

Actualmente la lógica de un solo travel_count vive en:

1. **App Android** — override de destino por sucursal, ajuste de +15 min, generación de ID
2. **PHP** — SQL directo, llamada a `buscar_nomina`, update de discrepancias
3. **Trigger PostgreSQL (Ruby)** — `clv_servicio`, `viaje_encontrado`, `ttpn_booking_id`

Cualquier cambio de regla requiere tocar los 3 lugares. Con Rails API, todo queda en Rails + trigger.

4.3 Deuda técnica acumulándose

PHP tiene 257+ endpoints (archivos `.php` individuales). Cada uno es un archivo de 20-150 líneas con SQL directo. No hay:

- Tests
- Documentación
- Versionado de cambios (git history mínimo)
- Estructura MVC
- Reutilización de código

El costo de mantener esto crece con cada nueva feature.

---

5. Qué se gana al eliminar PHP

Para el equipo de backend

- Una sola fuente de verdad para todas las reglas de negocio
- Tests automatizados posibles
- Deployments más confiables (sin sincronizar dos servidores)
- Refactoring seguro con cobertura de tests

Para el equipo móvil

- API REST estándar con documentación (Swagger/OpenAPI ya configurado en el proyecto con Rswag)
- Respuestas JSON consistentes con estructura predecible
- Errores descriptivos con campo `errors[]` en vez de mensajes de texto planos
- Un solo BASE_URL que cambia para actualizar toda la lógica

Para el negocio

- Auditoría completa de quién creó/modificó cada registro
- Control de acceso por rol (chofer vs capturista vs admin)
- Base para features nuevas sin deuda técnica
- Reducción de superficie de ataque (un servidor menos con credenciales de BD)

---

6. Riesgos de NO migrar

Riesgo	Probabilidad	Impacto
Colisión de IDs en INSERT simultáneos	Media	Alto — datos corruptos silenciosamente
Inyección SQL desde app comprometida	Media	Crítico — BD de producción expuesta
Bug de doble mantenimiento crea inconsistencia en nómina/cuadre	Alta	Alto — cuadre falla sin causa aparente
PHP server caído sin respaldo = app móvil inoperante	Alta	Crítico — operación detenida
Credenciales de BD filtradas si servidor PHP es comprometido	Baja	Crítico — pérdida total de datos

---

7. Plan de acción

Ver `PLAN_MIGRACION_MOVIL.md` para el plan en 2 fases con checklists y tiempos estimados.

Fecha objetivo para deprecar PHP: Al finalizar Fase 2 de migración móvil.
Prerrequisito: Todos los endpoints de la app móvil migrados a Rails API.

Conclusión

PHP es una deuda técnica que genera riesgos de seguridad reales y costos de mantenimiento crecientes. Ruby on Rails ya tiene todo lo necesario para sustituirlo: tablas, triggers, funciones, autenticación, y

estructura de API. La migración no es construir algo nuevo — es conectar la app móvil al backend que ya existe.